Celeste bootleg

Problem

This semestral work was focused on learning to work with the pygame module for python, and then recreating the Celeste PICO-8 mini game with it.

Used methods

The only notable method was the use of pygame. Simply put, it's a module that adds a bunch of functions and classes that help render a window, render rectangles inside of this window and then move them around. I found most use in pygame classes rect (rectangle), sprite, image, Surface, display. All of these handle objects, their dimensions, locations, textures and then their drawing onto some display. Other than that, I used the joystick class, to be able to make a gamepad communicate with the program, and have it as an alternate input device. The rest are mostly calculations and conditions. I also used the module pyautogui to generate automatically pressed keys for testing.

Results

Overall, I am very pleased with the result. When I started this project, the pygame library seemed like a lot to take in, and it is, but then I found a bunch of beginner guides and things started to make sense. It is safe to say that at the beginning I was very pessimistic about how the result would turn out.

I came across many problems during development, but these are the most noteworthy:

- Pythons dynamic approach to variable types. What I thought to be an integer became a float, float imprecisions during calculations took effect and my objects were way off.
- The outside desk and computer image. It is a fullHD picture with transparency in the middle. Rendering this 60x per second made the program run very slow, so I had to create a different screen for the game and for the picture, which now only renders once per game.
- The use of numpy, pandas or matplotlib. It was mandatory, but I couldn't find a good use for it, only some ways of forcing it in there. Thankfully the game is big enough to where I no longer had to fulfill this requirement.
- Key pressed vs. event. I used key pressed for the player movement. This looks if a key is pressed and if yes, returns True. In the game loop it does this check about 60x per second. In the menus this became a problem though. I fixed this using event KEYDOWN, which only returns a value once, when they key is pressed.

Further development

Some things I would add or change in the future:

- Hitboxes: currently, hitbox is as big as the animation image. This is a problem as some images are smaller and some bigger. It creates unwanted interactions with other objects, so ideally, I would have a separate rectangle for the characters hitbox.
- Levels as .txt files are very hard to work with. Find some other way of doing this.
- The game lags sometimes and I don't know why. I tried reverting commits when it started but that didn't help.

- Left/right movement momentum. Adding this would allow for more movement options and more interesting levels.
- I load an image for every block there is. Rework this to every block having a reference to the image, that is only loaded once.

These changes weren't implemented as I already spent 60+ hours on this work.

PEP8

Unfortunately, I couldn't meet this requirement. Two of the problems were pylint saying: no-member and c-extension-no-member. This message may report object members that are created dynamically, but exist at the time they are accessed. As this isn't a problem I caused and could fix, I decided to add # pylint: disable=no-member and # pylint: disable=c-extension-no-member to the beginning of affected files.

Other issues I ran into were:

- Too many instance attributes. I simply need all the attributes that the player and level classes have, and while I could make 5 sub-classes that only have attributes it would add more assigning, make code less readable, and pylint would be unhappy about those attribute classes having too few public methods. Thus, I decided to leave it as is.
- Global variables, but again, I simply needed to access these things located in the setting.py file across the whole program, and this seems like the best solution.
- Too few public methods. This happens on the Tile class in tile.py. I need this Sprite inherited class, and have no use for more public methods.

Tests

There are 3 tests that make sure calculation-based functions work as intended. The last, 4th test, is focused on testing if the games movement is deterministic and if it works as intended.

Unfortunately, the 4th test fails often. Which is the reason I marked it as xfail for pytest to expect it to fail. Explanation:

- One reason that it fails are the aforementioned lags. The test and game are 2 different parallel processes. When game lags, the test doesn't. The inputs get desynced and the result is way off.
- Game loop is different from the test loop. As I mentioned, these are 2 parallel processes, and sometimes one takes slightly longer to load than the other, which offsets the timings and the results vary slightly. These variations are usually by 8 pixels on x axis, which is equal to a single movement input that happens 60x per second. Sometimes the test ends before the last input and sometimes it doesn't. Which is how this difference happens.

A solution would be to create my own pyautogui module inside the game itself. I would be integrated into the game loop and thus lags would also affect the tests and timing offsets wouldn't happen.

Sources

- pygame documentation [online]. Open-source. Accessible via: https://www.pygame.org/docs/
- pyautogui documentation [online]. Al Sweigart Revision, 2019©, Accessible via: https://pyautogui.readthedocs.io/en/latest/